

D2Q9 · LBM · MPI + OpenMP

Optimisation hybride MPI + OpenMP d'un solveur LBM D2Q9

Allée de Kármán · $Re = 96$ · 800×160 cellules

Etude de performance d'un solveur hybride pour la simulation du sillage de Kármán.

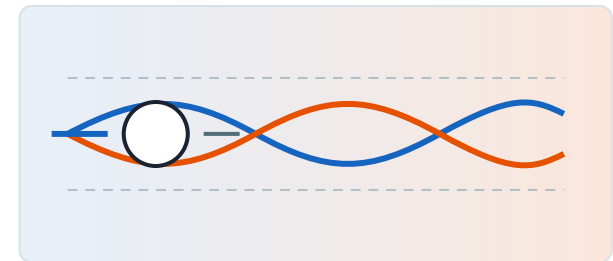
Résultat clé

Meilleur résultat mesuré : 2047 MLUPS

$np=1$, $OMP=16$

Cas d'étude

Canal 2D avec cylindre et sillage périodique



Domaine : 800×160 cellules · Régime : $Re = 96$

D2Q9 Lattice Boltzmann

Vortex de Kármán · $Re = 96$
Domaine : 800×160 cellules
20 000 pas de temps



Modèle D2Q9 : centre + 8 directions
9 vitesses discrètes par cellule

Figure de mérite : MLUPS

$$MLUPS = W \times H \times N / (T \times 10^6)$$

Millions de Lattice Updates Par Seconde

$$W = 800 \cdot H = 160 \cdot N = 20\,000$$

$$800 \times 160 \times 20\,000 = 2.56 \times 10^9 \text{ cellules}$$

Temps de calcul T mesuré en secondes

Plus MLUPS est grand, plus c'est performant

Objectif : maximiser le débit de calcul

Référence initiale : 60 MLUPS (macOS AArch64, avant corrections)

Partie 1 — macOS AArch64 · opt01–17

60

103

194

216

Partie 2 — AMD Ryzen 9 x86-64 · opt18–26

832

2047

Partie 1 — macOS AArch64

opt01–03 : Validité benchmark · 60 MLUPS

opt04–08 : Latence MPI réduite · 103 MLUPS

opt09–13 : Pull + OpenMP · 194 MLUPS

opt14–17 : Diagnostics, ex. SIMD · gains Apple limités

Jalons retenus : 60 → 103 → 194 MLUPS

Partie 2 — AMD Ryzen 9 x86-64

opt18–19 : AoS → SoA · 216 MLUPS

opt20–22 : AVX-512 · 832 MLUPS

opt23–26 : Halo, OMP, protocole · 2047 MLUPS

Jalons retenus : 216 → 832 → 2047 MLUPS

Gain global indicatif · plateformes différentes · non iso-plateforme

opt01 — Suppression du nanosleep caché

nanosleep() encodé dans un template CMake — invoqué implicitement à chaque halo_exchange.

Impact : 5,5 h perdues par simulation de 20 000 pas. Faussait toutes les mesures.

opt02 — Correction de la formule MLUPS

Formule surestimait le FOM d'un facteur 4 en mode multi-processus.

Impact : les mesures multi-rang semblaient plus rapides — masquait le coût MPI réel.

opt03 — Correction des tailles MPI send/recv

Incohérence entre les tailles d'envoi et de réception dans halo_exchange — corrigée.

Évite des échanges halo erronés et des comportements indéfinis sur implémentations MPI strictes.

Référence validée : 60 MLUPS (np=1, OMP=1)

12 barrières supprimées · 80 → 1 horizontal · 798 → 1 vertical

Insight clé : le coût MPI vient de la latence de démarrage de chaque appel, pas du volume de données

Avant (opt03)

Envois maille-par-maille
7 182 messages / itération
MPI_Barrier × 12 par itération
Synchronisation globale bloquante

68,7 MLUPS

np=2, OMP=1

Latence cumulée dominante



Après (opt04–08)

Envoi groupé par ligne halo
1 message / direction
Barrières redondantes supprimées
Volume de données inchangé

102,75 MLUPS

np=2, OMP=1

Latence fortement réduite

Jalon P1-B : 103 MLUPS · +50%

Push (avant)

La source écrit vers sa cible
Plusieurs threads peuvent écrire
sur la même cellule cible.

```
mesh_out[i+dx][j+dy] = ...  
→ écriture conflict (race)
```

Race condition sans mutex
Non parallélisable sans verrou

OpenMP requière des sections
critiques → sérialisation

Schéma Pull

La cible lit depuis sa source
Chaque thread est propriétaire
de sa cellule de destination.

```
mesh_out[i][j] = mesh_in[i-dx][j-dy]  
→ écriture exclusive
```

Race-free, sans verrou
Chaque thread isolé en écriture

Équivalence mathématique
garantie par symétrie D2Q9

OpenMP

Pull rend le parallélisme trivial

```
#pragma omp parallel for  
schedule(static)  
for (i = 0; i < width; i++)
```

Colonnes réparties entre threads
Collision + propagation parallèles

Localité cache :

j interne, stride-1
accès contigu en mémoire

OMP=4 : x4 threads actifs
sans section critique

Certaines pistes testées n'ont pas été retenues, mais elles ont clarifié les choix techniques.

SIMD testé

```
-march=native  
#pragma omp simd
```

Hypothèse :
plus de SIMD → plus rapide

Plateforme cible :
macOS AArch64
Apple M3 (P/E cores)

-O3 génère déjà NEON 2xdouble
(fmla.2d – visible à l'objdump)

Résultat macOS

-O3 seul vs -O3 -march=native + simd

np=1, OMP=1
62 → 54 MLUPS (-13%)

np=1, OMP=4
135 → 121 MLUPS (-10%)

Diagnostic probable : lectures non contiguës
+ ordonnanceur P/E cores.

Décision

- Conserver -O3 sur macOS
- Ne pas généraliser -march=native
- AVX-512 traité séparément sur AMD (Partie 2)
- Aucun changement de code (analyse et décision seules)

-O3 suffit sur AArch64 :
fmla.2d confirmé (objdump)

Autres travaux P1-D :

overlap MPI · fusion collision · I/O distribué

Rôle :

diagnostic · portabilité · préparation pour la suite

Plateforme

AMD Ryzen 9 8940HX

Zen4 · 16c/32t · AVX-512

GCC 15.2.1 · MPICH 4.3

Linux x86-64

AVX-512 : 8 doubles par registre zmm

Potentiel SIMD maximal pour LBM

Précédente : macOS AArch64

Apple M · NEON SIMD · Clang

P1 : opt01–17 validées jusqu'à 194 MLUPS

Plateformes non comparables directement

Gain global indicatif, non iso-plateforme

Protocole unifié

- ✓ 1 warm-up + 20 répétitions · médiane
- ✓ CPU fréq. verrouillée (cpupower)
- ✓ drop_caches avant chaque mesure
- ✓ PROFILING=OFF
- ✓ Répétitions médianes (non moyenne)

Référence P2 dev : 216 MLUPS
valeur de développement

np=2, OMP=4

AoS — Array of Structures

```
cell[i][j].f[k]
```

stride 72 oct pour le même k entre deux cellules voisines



Saut de 72 oct pour atteindre f[k] de la cellule suivante

Non vectorisable : stride > 1

Le compilateur ne peut pas charger un vecteur zmm
depuis des adresses non contiguës



SoA — Structure of Arrays

```
cells[k][x*H+y]
```

cellules adj. en j, à k fixé : contiguës



8 doubles contigus → 1 chargement zmm

Vectorisable : stride 1 en j

prérequis structurel pour AVX-512

La contiguïté en j par plan k permet la vectorisation SIMD

Régression initiale -22% — investissement nécessaire pour +285%

opt20 — k-outer

Boucle k mise en extérieur
Accès par plans plus réguliers
Meilleure localité mémoire

216 → 319 MLUPS

Étape de développement

opt21 — multicell

Kernel multicell
Structure de boucle réorganisée
Prépare la vectorisation explicite

319 → 423 MLUPS

Étape intermédiaire

opt22 — __restrict__ + ivdep

__restrict__ : pas d'alias
#pragma GCC ivdep
Levée d'analyse d'alias

423 → 832 MLUPS

GCC émet des registres zmm

Preuve assembleur (objdump) :

```
vmovupd (%r10,%rax,1), %zmm23  
vaddpd / vdivpd – registres zmm confirmés
```

Chaîne de développement : 216 → 832 MLUPS

832 mesuré np=2, OMP=4 · 757 mesuré np=1, OMP=4

Valeurs de développement — non protocole final — résultats officiels sur slide suivant

opt24 · Halo k-major

Packing halo réorganisé en k-major

Accès mémoire plus réguliers

Localité améliorée pour pack/unpack

Réorganisation cohérente avec le layout SoA

832 → 905 MLUPS

(donnée de développement)

(non protocole final)

opt25 · Diagnostic Waitall

Arrivées déséquilibrées entre ranks

Diagnostic intra-nœud : attente surtout

due au déséquilibre entre ranks

Diagnostic seul

opt26 · Région OMP unique

#pragma omp parallel

Une région OpenMP regroupe

les phases d'un même timestep

gain limité · structure plus propre

Architecture propre

Valeurs de développement (sans verrou CPU) · résultats officiels sur slide suivant

np	OMP	MLUPS	Note
1	4	898	—
1	16	2047	meilleur ★
2	4	1071	★ multi-rang
2	8	870	—
4	4	535	surcharge MPI

1 warm-up + 20 rép. · médiane · CPU verrouillé

2047

MLUPS

np=1, OMP=16

1071 MLUPS (np=2, OMP=4)
meilleur résultat multi-rang

Réf. 60 MLUPS sur macOS AArch64
gain global indicatif, non iso-plateforme

Leçons clés

✓ Latence MPI > volume

Fusion de messages : 7182 → 1 msg/dir
+50% (68.7 → 102.75 MLUPS, np=2 OMP=1)

✓ SoA = régression initiale

Investissement structurel nécessaire
pour débloquent AVX-512

✓ __restrict__ + ivdep : analyse d'alias

Clé pour émission des registres zmm
+97% à cette seule étape (423 → 832)

Questions ouvertes

? MPI inter-nœuds

Impact réel du halo sur réseau
Latence réseau vs. déséquilibre de rangs ?

? Validation physique

Strouhal, lift/drag quantitat. ?
Comparaison avec valeurs théoriques

? Passage D3Q19, GPU

Extension 3D (D3Q19)
GPU : CUDA/HIP pour accélération massive

2047 MLUPS · meilleur résultat mesuré · 2 plateformes · 26 optimisations

Merci · Questions bienvenues